

Alumno/a:	Paula Martínez Lechón	NIA:	100522137
Alumno/a:	Xiya Ye	NIA:	100522159
Alumno/a:	Marta Vindel Navas	NIA:	100522119

1 Introducción

Esta práctica consiste en trabajar con el diseño físico de una base de datos en ORACLE, enfocándose sobre todo en analizar el funcionamiento del sistema y aplicar mejoras para optimizar su rendimiento.

Para ello, partimos de un entorno de pruebas basado en la práctica anterior y un script que contiene un paquete con que devuelve las estadísticas necesarias para su evaluación. Ejecutando el paquete, se puede ver que partimos de un diseño físico que, al realizar las consultas dadas, necesita 60.525 accesos. Nuestro objetivo en esta práctica será intentar disminuir al máximo este número.

```
SQL> exec pkg_costes.run_test(10);  
Iteration 1  
Iteration 2  
Iteration 3  
Iteration 4  
Iteration 5  
Iteration 6  
Iteration 7  
Iteration 8  
Iteration 9  
Iteration 10  
RESULTS AT 01/05/2025 16:33:22  
TIME CONSUMPTION (run): 1109.8 milliseconds.  
CONSISTENT GETS (workload):60525 acc  
CONSISTENT GETS (weighted average):6052.5 acc  
  
PL/SQL procedure successfully completed.
```

Estadísticas del diseño original

A lo largo del trabajo, se analizará el coste de ciertas consultas con la configuración inicial, se estudiarán los planes de ejecución y se propondrá un nuevo diseño físico que reduzca el número de accesos a memoria mediante la creación de índices, clusters y ajustes de parámetros físicos como el PCTFREE. Finalmente, se volverá a ejecutar la carga de trabajo para comparar el rendimiento del diseño físico original con el diseño mejorado.

2 Análisis

Nuestro diseño físico actual sigue lo establecido por defecto en ORACLE, es decir, tiene una estructura serial no consecutiva con un espacio de bloque de 8KB, un espacio libre distribuido (PCTFREE) del 10%, una ocupación mínima de cubo (PCTUSED) del 60% y el espacio ocupado por los índices de las claves primarias.

Como se ha indicado en la introducción, nuestro objetivo es mejorar la eficiencia de la base de datos. Hemos podido observar que una de las cosas que influyen en la baja eficiencia es que ninguna de las claves que usamos para filtrar es una clave primaria, lo que en una organización serial implica que se recurre al *full scan* y se necesitan más accesos.

A continuación analizaremos cuatro consultas, que son, según lo que se ha estudiado, las más recurrentes. Estas consultas tienen unas frecuencias relativas de {0.3, 0.3, 0.3, 0.1}, tomadas en cuenta a la hora de pasar los tests proporcionados en el paquete. Es por ello que usamos los valores proporcionados en los tests para pensar en un diseño físico más óptimo.

- **Consulta 1:** `select * from editions where pub_place='...';`

En la primera consulta nos encontramos con una búsqueda sobre la tabla 'Editions' en la que seleccionamos filas en base a su lugar de publicación. En el análisis que vamos a realizar, se estudian tres lugares, por lo que la cantidad de filas y los accesos a memoria obtenidos varían según el dato introducido.

```
SQL> desc editions;
```

Name	Null?	Type
-----	-----	-----
ISBN	NOT NULL	VARCHAR2(20)
TITLE	NOT NULL	VARCHAR2(200)
AUTHOR	NOT NULL	VARCHAR2(100)
LANGUAGE	NOT NULL	VARCHAR2(50)
ALT_LANGUAGES		VARCHAR2(50)
EDITION		VARCHAR2(50)
PUBLISHER		VARCHAR2(100)
EXTENSION		VARCHAR2(50)
SERIES		VARCHAR2(50)
COPYRIGHT		VARCHAR2(20)
PUB_PLACE		VARCHAR2(50)
DIMENSIONS		VARCHAR2(50)
PHY_FEATURES		VARCHAR2(200)
MATERIALS		VARCHAR2(200)
NOTES		VARCHAR2(500)
NATIONAL_LIB_ID	NOT NULL	VARCHAR2(20)
URL		VARCHAR2(200)

Descripción de la tabla 'Editions'

Plan de ejecución de la consulta con pub_place='Madrid':

```
SQL> select * from editions where pub_place='Madrid'
2
;
82450 rows selected.

Execution Plan
-----
Plan hash value: 1741989577

-----
| Id | Operation          | Name          | Rows  | Bytes | Cost (%CPU)| Time     |
-----
| 0  | SELECT STATEMENT   |               |      1 |      |    255 (1)| 00:00:01 |
|* 1 | TABLE ACCESS FULL| EDITIONS      | 82926 | 17M   |    255 (1)| 00:00:01 |
-----

Predicate Information (identified by operation id):
-----
1 - filter("PUB_PLACE"='Madrid')

Statistics
-----
1 recursive calls
0 db block gets
12897 consistent gets
0 physical reads
0 redo size
17925155 bytes sent via SQL*Net to client
60830 bytes received via SQL*Net from client
5498 SQL*Net roundtrips to/from client
0 sorts (memory)
0 sorts (disk)
82450 rows processed
```

Plan de ejecución de select * from editions where pub_place='Madrid';

En este caso el número de filas obtenido es considerablemente alto, lo que afecta a las posibles mejoras del diseño. Debido a esta alta tasa de coincidencia el sistema realizará un *full scan*, puesto que puede aprovechar los beneficios de la lectura física. Por tanto, la creación de un índice empeoraría la eficiencia de la búsqueda, por lo que, a no ser que se fuerce con una hint, lo ignoraría.

No obstante, se podrían hacer otros cambios, como la creación de un cluster, que reduzcan el coste de la consulta. Esto se debe a que ORACLE y, por consiguiente, todas las tablas de la base de datos, utilizan una organización base serial, lo que quiere decir que los datos están desordenados. Mientras tanto, el cluster se ha creado en base a 'pub_place', lo que provoca que los datos estén ordenados por esa clave. En este caso, al coincidir la clave de búsqueda con la del cluster, se puede seguir aprovechando los beneficios de físicos sin tener que recurrir a un *full scan*, por lo que el coste de la consulta será menor que haciendo la búsqueda directamente sobre la tabla original. De no coincidir las claves el coste sería el mismo.

Plan de ejecución de la consulta con pub_place='Segovia':

```
SQL> select * from editions where pub_place='Segovia';
71 rows selected.

Execution Plan
-----
Plan hash value: 1741989577

-----
| Id | Operation          | Name          | Rows  | Bytes | Cost (%CPU)| Time     |
-----
| 0  | SELECT STATEMENT   |               |      1 |      |    254 (1)| 00:00:01 |
|* 1 | TABLE ACCESS FULL| EDITIONS      | 133   | 29127 |    254 (1)| 00:00:01 |
-----

Predicate Information (identified by operation id):
-----
1 - filter("PUB_PLACE"='Segovia')

Statistics
-----
1 recursive calls
0 db block gets
7566 consistent gets
0 physical reads
0 redo size
16895 bytes sent via SQL*Net to client
418 bytes received via SQL*Net from client
6 SQL*Net roundtrips to/from client
0 sorts (memory)
0 sorts (disk)
71 rows processed
```

Plan de ejecución de select * from editions where pub_place='Segovia';

Como en este caso el número de filas devueltas no es tan alto, podemos considerar el uso de índices para evitar que realice un *full scan*. En este caso, a diferencia del anterior, sí que se utilizaría el índice, ya que la baja tasa de coincidencia hace que sea más eficiente que el *full scan* incluso teniendo en cuenta los beneficios de la lectura física.

Para esta consulta añadir un cluster (sin el índice) también disminuiría los accesos a memoria. Como se ha mencionado anteriormente, los datos en el cluster estarían ordenados, por lo que también mejoraría la búsqueda, al no tener que recurrir al *full scan*.

Plan de ejecución de la consulta con `pub_place='Barataria'`:

```
SQL> select * from editions where pub_place='Barataria';
no rows selected

Execution Plan
-----
Plan hash value: 1741989577

-----
| Id | Operation          | Name      | Rows  | Bytes | Cost (%CPU)| Time     |
-----
| 0  | SELECT STATEMENT   |           |      10 | 2190 | 2054 (1)| 00:00:01 |
|* 1 | TABLE ACCESS FULL | EDITIONS  |      10 | 2190 | 2054 (1)| 00:00:01 |
-----

Predicate Information (identified by operation id):
-----
 1 - filter("PUB_PLACE"='Barataria')

Statistics
-----
 1 recursive calls
 0 db block gets
 7562 consistent gets
 0 physical reads
 0 redo size
 1467 bytes sent via SQL*Net to client
 365 bytes received via SQL*Net from client
 1 SQL*Net roundtrips to/from client
 0 sorts (memory)
 0 sorts (disk)
 0 rows processed
```

Plan de ejecución de `select * from editions where pub_place='Barataria'`;

Como podemos observar en la imagen, esta consulta no devuelve ninguna fila, por lo que tendrá que recorrer todo el espacio sobre el que realice la búsqueda. Un índice sobre 'pub_place' será de un tamaño mucho menor que la tabla, con lo que se reducirá el coste de la consulta con respecto a un *full scan*.

Otra opción que también provoca una mejora es la creación de un cluster sobre 'pub_place', puesto que, al estar ordenado por la clave de búsqueda, se mejora el coste de la consulta.

- **Consulta 2:** `select * from editions where publisher='...';`

En esta segunda consulta continuamos trabajando sobre la tabla 'Editions', pero esta vez seleccionamos las filas en base a la editorial. En el análisis que vamos a realizar, se estudian tres editoriales, por lo que, al igual que la consulta anterior, la cantidad de filas y los accesos a memoria obtenidos varían según el dato introducido.

Plan de ejecución de las consultas con publisher='B' y publisher='SM':

<pre>SQL> select * from editions where publisher='B'; 12 rows selected. Execution Plan ----- Plan hash value: 1741989577 +-----+-----+-----+-----+-----+-----+ Id Operation Name Rows Bytes Cost (%CPU) Time +-----+-----+-----+-----+-----+-----+ 0 SELECT STATEMENT * 1 TABLE ACCESS FULL EDITIONS 8 1752 2053 (1) 00:00:01 +-----+-----+-----+-----+-----+-----+ Predicate Information (identified by operation id): ----- 1 - filter("PUBLISHER"='B') Statistics ----- 22 recursive calls 0 db block gets 7622 consistent gets 7570 physical reads 0 redo size 3224 bytes sent via SQL*Net to client 368 bytes received via SQL*Net from client 2 SQL*Net roundtrips to/from client 0 sorts (memory) 0 sorts (disk) 12 rows processed</pre>	<pre>SQL> select * from editions where publisher='SM'; 1704 rows selected. Execution Plan ----- Plan hash value: 1741989577 +-----+-----+-----+-----+-----+-----+ Id Operation Name Rows Bytes Cost (%CPU) Time +-----+-----+-----+-----+-----+-----+ 0 SELECT STATEMENT * 1 TABLE ACCESS FULL EDITIONS 1320 284K 2053 (1) 00:00:01 +-----+-----+-----+-----+-----+-----+ Predicate Information (identified by operation id): ----- 1 - filter("PUBLISHER"='SM') Statistics ----- 1 recursive calls 0 db block gets 7675 consistent gets 7552 physical reads 0 redo size 323620 bytes sent via SQL*Net to client 1612 bytes received via SQL*Net from client 115 SQL*Net roundtrips to/from client 0 sorts (memory) 0 sorts (disk) 1704 rows processed</pre>
---	--

Plan de ejecución de
select * from editions where publisher='B';

Plan de ejecución de
select * from editions where publisher='SM';

En estas capturas, se puede ver que ambas consultas devuelven un número de filas bastante bajo, por lo que se tratan de casos muy parecidos a la consulta anterior donde pub_place='Segovia'. Por tanto, podemos considerar las mismas opciones para optimizar estas consultas: crear un índice o crear un cluster.

Plan de ejecución de la consulta con publisher='C':

```
SQL> select * from editions where publisher='C';
no rows selected

Execution Plan
-----
Plan hash value: 1741989577

+-----+-----+-----+-----+-----+-----+
| Id | Operation | Name | Rows | Bytes | Cost (%CPU)| Time |
+-----+-----+-----+-----+-----+-----+
| 0 | SELECT STATEMENT | | | | | |
|* 1 | TABLE ACCESS FULL | EDITIONS | 8 | 1752 | 2053 (1)| 00:00:01 |
+-----+-----+-----+-----+-----+-----+

Predicate Information (identified by operation id):
-----
1 - filter("PUBLISHER"='C')

Statistics
-----
1 recursive calls
0 db block gets
7562 consistent gets
7552 physical reads
0 redo size
1467 bytes sent via SQL*Net to client
357 bytes received via SQL*Net from client
1 SQL*Net roundtrips to/from client
0 sorts (memory)
0 sorts (disk)
0 rows processed
```

Plan de ejecución de select * from editions where publisher='C';

En esta consulta sucede algo similar a la primera consulta con pub_place='Barataria', que no devuelve ninguna fila. Por tanto, por los mismos motivos, sabemos que la creación de un índice o de un cluster mejorarán el coste.

- **Consulta 3:** select * from copies where condition='...';

En esta tercera consulta realizamos las búsquedas sobre la tabla 'Copies', seleccionando las filas de esta en base a su condición. Para analizar esta consulta, se han usado las tres condiciones propuestas en los tests.

```
SQL> desc copies;
```

Name	Null?	Type
SIGNATURE	NOT NULL	CHAR(5)
ISBN	NOT NULL	VARCHAR2(20)
CONDITION	NOT NULL	CHAR(1)
COMMENTS		VARCHAR2(20)
DEREGISTERED		DATE

Descripción de la tabla 'Copies'

En este caso, los resultados de las consultas son muy similares, por lo que no es necesario analizarlas por separado.

Plan de ejecución de las consultas con condition='G', condition='N' y condition='D':

```
SQL> select * from copies where condition='G';
```

48265 rows selected.

Execution Plan

Plan hash value: 468958009

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		48265	1131K	278 (3)	00:00:01
* 1	TABLE ACCESS FULL	COPIES	48265	1131K	278 (3)	00:00:01

Predicate Information (identified by operation id):

1 - filter("CONDITION"='G')

Statistics

16	recursive calls
0	db block gets
4229	consistent gets
1000	physical reads
0	redo size
1884500	bytes sent via SQL*Net to client
35753	bytes received via SQL*Net from client
3219	SQL*Net roundtrips to/from client
0	sorts (memory)
0	sorts (disk)
48265	rows processed

Plan de ejecución de
select * from copies where condition='G';

```
SQL> select * from copies where condition='N';
```

48404 rows selected.

Execution Plan

Plan hash value: 468958009

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		48404	1134K	278 (3)	00:00:01
* 1	TABLE ACCESS FULL	COPIES	48404	1134K	278 (3)	00:00:01

Predicate Information (identified by operation id):

1 - filter("CONDITION"='N')

Statistics

8	recursive calls
0	db block gets
4203	consistent gets
0	physical reads
0	redo size
1889715	bytes sent via SQL*Net to client
35852	bytes received via SQL*Net from client
3228	SQL*Net roundtrips to/from client
0	sorts (memory)
0	sorts (disk)
48404	rows processed

Plan de ejecución de
select * from copies where condition='N';

```
SQL> select * from copies where condition='D';
```

48466 rows selected.

Execution Plan

Plan hash value: 468958009

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		48466	1135K	278 (3)	00:00:01
* 1	TABLE ACCESS FULL	COPIES	48466	1135K	278 (3)	00:00:01

Predicate Information (identified by operation id):

1 - filter("CONDITION"='D')

Statistics

8	recursive calls
0	db block gets
4213	consistent gets
0	physical reads
0	redo size
1892318	bytes sent via SQL*Net to client
35907	bytes received via SQL*Net from client
3233	SQL*Net roundtrips to/from client
0	sorts (memory)
0	sorts (disk)
48466	rows processed

Plan de ejecución de select * from copies where condition='D';

Como se puede ver, estas tres consultas devuelven un número de filas muy alto, por lo que tienen unas características muy parecidas a la primera consulta con `pub_place='Madrid'`. Por tanto, la creación de un índice no optimizaría los accesos para la búsqueda, así que la opción ideal sería hacer un *full scan*.

- **Consulta 4:** `select * from editions;`

En la última consulta volvemos a realizar una búsqueda sobre la tabla 'Editions', pero esta vez no se realiza con ninguna condición de búsqueda.

Plan de ejecución

```
SQL> select * from editions;
240632 rows selected.

Execution Plan
-----
Plan hash value: 1741989577

-----
| Id | Operation          | Name      | Rows  | Bytes | Cost (%CPU)| Time     |
-----
| 0  | SELECT STATEMENT   |           | 240K  | 50M   | 2055 (1)| 00:00:01 |
| 1  | TABLE ACCESS FULL| EDITIONS  | 240K  | 50M   | 2055 (1)| 00:00:01 |
-----

Statistics
-----
1 recursive calls
0 db block gets
23110 consistent gets
7552 physical reads
0 redo size
57147402 bytes sent via SQL*Net to client
176810 bytes received via SQL*Net from client
16044 SQL*Net roundtrips to/from client
0 sorts (memory)
0 sorts (disk)
240632 rows processed
```

Plan de ejecución de `select * from editions;`

En esta consulta, al tratarse de una consulta a toda la tabla, la creación de índices o de clusters no optimiza la búsqueda, ya que aunque esté ordenado por un índice o un cluster, siempre debe leer la tabla al completo. Por tanto, es más preferible hacer un *full scan*.

No obstante, sí sería posible reducir el coste modificando los parámetros físicos de la base de datos, como por ejemplo, reducir el espacio libre distribuido (PCTFREE) de los cubos. Puesto que se trata de una consulta a la totalidad, cuanto menos espacio libre haya, menos accesos va a necesitar. Al estar este espacio reservado para futuras modificaciones, que no se realizan en este caso, no es necesario contar con él. Además, al reducir el PCTFREE, se aprovecha mejor el espacio del dispositivo, pudiendo almacenar más registros por cubo y aumentando la densidad del modelo. Por tanto, es recomendable disminuir el espacio libre distribuido para optimizar esta consulta, siempre y cuando no se realicen muchas actualizaciones en la tabla.

3 Diseño Físico

En este apartado miramos cómo afectan los cambios propuestos en el apartado anterior a las distintas consultas. En función de los *consistent gets* valoramos si las modificaciones mejoran o no el diseño original.

- **Consulta 1:** select * from editions where pub_place='...';

Creación de un índice:

Como se ha mencionado anteriormente, a pesar de que para el caso en el que el pub_place='Madrid' el índice no es útil, para los otros dos casos hay una mejora importante. Es por ello que hemos decidido crear un índice. Para crear el índice se ha usado el siguiente código:

```
create index idx_ed1 on editions(pub_place);
```

Creación de un cluster:

En otro caso, se creó un cluster para comprobar si el resultado era mejor, igual o peor que usando un índice, para después poder hacer la correspondiente comparación, ya que para el caso de pub_place='Madrid' parece que el cluster es más adecuado para la optimización. Para ello se usó el siguiente código:

```
--
create cluster places(pub_place varchar2(50));

CREATE TABLE Editions(
ISBN          VARCHAR2(20),
TITLE         VARCHAR2(200) NOT NULL,
AUTHOR        VARCHAR2(100) NOT NULL,
LANGUAGE      VARCHAR2(50) default('Spanish') NOT NULL,
ALT_LANGUAGES VARCHAR2(50),
EDITION       VARCHAR2(50),
PUBLISHER     VARCHAR2(100),
EXTENSION     VARCHAR2(50),
SERIES        VARCHAR2(50),
COPYRIGHT     VARCHAR2(20),
PUB_PLACE     VARCHAR2(50),
DIMENSIONS    VARCHAR2(50),
PHY_FEATURES  VARCHAR2(200),
MATERIALS     VARCHAR2(200),
NOTES         VARCHAR2(500),
NATIONAL_LIB_ID VARCHAR2(20) NOT NULL,
URL           VARCHAR2(200),
CONSTRAINT pk_editions PRIMARY KEY(isbn),
CONSTRAINT uk_editions UNIQUE (national_lib_id),
CONSTRAINT fk_editions_books FOREIGN KEY(title,author)
REFERENCES books(title,author)
) cluster places(pub_place);

create index idx_places on cluster places;
```


A continuación, comprobamos si se produjeron mejoras en los planes de ejecución de las consultas, ya sea usando el índice o bien el cluster:

Plan de ejecución de la consulta con `pub_place='Madrid'`:

```
SQL> select /*+ index(editions) */ * from editions where pub_place='Madrid';
82450 rows selected.

Execution Plan
-----
Plan hash value: 4255503933

   Id | Operation                                | Name      | Rows  | Bytes | Cost (%CPU)| Time     |
---- | -
0    | SELECT STATEMENT                        |           |      1 |       |  32362 (1)| 00:00:01 |
1    | TABLE ACCESS BY INDEX ROWID BATCHED    | EDITIONS  | 82926 | 17M   | 32362 (1)| 00:00:01 |
* 2  | INDEX RANGE SCAN                        | IDX_ED1   |      1 |       |    254 (1)| 00:00:01 |

Predicate Information (identified by operation id):
-----
   2 - access("PUB_PLACE"='Madrid')

Statistics
-----
          0 recursive calls
          0 db block gets
       18224 consistent gets
        7716 physical reads
           0 redo size
    19595858 bytes sent via SQL*Net to client
       68852 bytes received via SQL*Net from client
        5498 SQL*Net roundtrips to/from client
           0 sorts (memory)
           0 sorts (disk)
       82450 rows processed
```

Usando un índice: plan de ejecución de
`select * from editions where pub_place='Madrid';`

```
SQL> select * from editions where pub_place='Madrid';
82450 rows selected.

Execution Plan
-----
Plan hash value: 9953560

   Id | Operation                                | Name      | Rows  | Bytes | Cost (%CPU)| Time     |
---- | -
0    | SELECT STATEMENT                        |           |      1 |       |  69410 (2)| 00:00:01 |
1    | TABLE ACCESS CLUSTER                  | EDITIONS  | 69410 | 65M   |  69410 (2)| 00:00:01 |
* 2  | INDEX UNIQUE SCAN                      | IDX_PLACES |      1 |       |         1 (0)| 00:00:01 |

Predicate Information (identified by operation id):
-----
   2 - access("PUB_PLACE"='Madrid')

Note
-----
   - dynamic statistics used: dynamic sampling (level=2)

Statistics
-----
          10 recursive calls
           1 db block gets
        8236 consistent gets
         288 physical reads
         184 redo size
    17923534 bytes sent via SQL*Net to client
       68829 bytes received via SQL*Net from client
        5498 SQL*Net roundtrips to/from client
           0 sorts (memory)
           0 sorts (disk)
       82450 rows processed
```

Usando un cluster: plan de ejecución de
`select * from editions where pub_place='Madrid';`

En este caso, como se mencionó en el análisis, SQL ignora el índice y realiza un *full scan*, puesto que, cuando forzamos su ejecución con una hint, observamos que la búsqueda es más costosa.

En cambio, con el cluster, sí se observa una clara mejora, ya que pasa de tener 12.897 *consistent gets* a tener 8.236. Vemos que empeora en la cantidad de *physical reads*, pero el cambio sigue mereciendo la pena.

Por lo tanto, podemos concluir que para este caso la mejor opción sería que se implementara un cluster. No obstante, tenemos que tener en cuenta también el resto de consultas.

Plan de ejecución de la consulta con `pub_place='Segovia'`:

```
SQL> select * from editions where pub_place='Segovia';
71 rows selected.

Execution Plan
-----
Plan hash value: 4255503933

   Id | Operation                                | Name      | Rows  | Bytes | Cost (%CPU)| Time     |
---- | -
0    | SELECT STATEMENT                        |           |      1 |       |  29127 (1)| 00:00:01 |
1    | TABLE ACCESS BY INDEX ROWID BATCHED    | EDITIONS  | 133   | 29127 |  29127 (1)| 00:00:01 |
* 2  | INDEX RANGE SCAN                        | IDX_ED1   |      1 |       |         3 (0)| 00:00:01 |

Predicate Information (identified by operation id):
-----
   2 - access("PUB_PLACE"='Segovia')

Statistics
-----
          0 recursive calls
          0 db block gets
           0 consistent gets
           0 physical reads
           0 redo size
    18569 bytes sent via SQL*Net to client
         418 bytes received via SQL*Net from client
           6 SQL*Net roundtrips to/from client
           0 sorts (memory)
           0 sorts (disk)
         71 rows processed
```

Usando un índice: plan de ejecución de
`select * from editions where pub_place='Segovia';`

```
SQL> select * from editions where pub_place='Segovia';
71 rows selected.

Execution Plan
-----
Plan hash value: 9953560

   Id | Operation                                | Name      | Rows  | Bytes | Cost (%CPU)| Time     |
---- | -
0    | SELECT STATEMENT                        |           |      1 |       |  129 (2)| 00:00:01 |
1    | TABLE ACCESS CLUSTER                  | EDITIONS  | 129   | 124K  |  129 (2)| 00:00:01 |
* 2  | INDEX UNIQUE SCAN                      | IDX_PLACES |      1 |       |         1 (0)| 00:00:01 |

Predicate Information (identified by operation id):
-----
   2 - access("PUB_PLACE"='Segovia')

Note
-----
   - dynamic statistics used: dynamic sampling (level=2)

Statistics
-----
           5 recursive calls
           0 db block gets
          96 consistent gets
           0 physical reads
           0 redo size
    16907 bytes sent via SQL*Net to client
         419 bytes received via SQL*Net from client
           6 SQL*Net roundtrips to/from client
           0 sorts (memory)
           0 sorts (disk)
         71 rows processed
```

Usando un cluster: plan de ejecución de
`select * from editions where pub_place='Segovia';`

En este caso, la mejora usando ambos es muy notoria, ya que pasamos de tener 7.566 *consistent gets* a 80 y 96 respectivamente. Esto se debe a que, siguiendo lo comentado anteriormente, no se debe recorrer toda la tabla para localizar pocas filas.

Sin embargo, podemos ver que la mejora con el cluster, pese a ser buena es peor que la mejora usando el índice.

Plan de ejecución de la consulta con `pub_place='Barataria'`:

```
SQL> select * from editions where pub_place='Barataria';
no rows selected

Execution Plan
-----
Plan hash value: 4255503933

-----
| Id | Operation | Name | Rows | Bytes | Cost (%CPU) | Time |
-----
| 0 | SELECT STATEMENT | | | | | |
| 1 | TABLE ACCESS BY INDEX ROWID BATCHED | EDITIONS | 10 | 2190 | 7 (0) | 00:00:01 |
| * 2 | INDEX RANGE SCAN | IDX_ED1 | 10 | | 3 (0) | 00:00:01 |
-----

Predicate Information (identified by operation id):
-----
 2 - access("PUB_PLACE"='Barataria')

Statistics
-----
 1 recursive calls
 0 db block gets
 3 consistent gets
 3 physical reads
 0 redo size
1467 bytes sent via SQL*Net to client
365 bytes received via SQL*Net from client
 1 SQL*Net roundtrips to/from client
 0 sorts (memory)
 0 sorts (disk)
 0 rows processed
```

Usando un índice: plan de ejecución de
`select * from editions where pub_place='Barataria';`

```
SQL> select * from editions where pub_place='Barataria';
no rows selected

Execution Plan
-----
Plan hash value: 9953560

-----
| Id | Operation | Name | Rows | Bytes | Cost (%CPU) | Time |
-----
| 0 | SELECT STATEMENT | | | | | |
| 1 | TABLE ACCESS CLUSTER | EDITIONS | 129 | 124K | 2 (0) | 00:00:01 |
| * 2 | INDEX UNIQUE SCAN | IDX_PLACES | 1 | | 1 (0) | 00:00:01 |
-----

Predicate Information (identified by operation id):
-----
 2 - access("PUB_PLACE"='Barataria')

Note
-----
- dynamic statistics used: dynamic sampling (level=2)

Statistics
-----
 25 recursive calls
 0 db block gets
111 consistent gets
 2 physical reads
 0 redo size
1467 bytes sent via SQL*Net to client
365 bytes received via SQL*Net from client
 1 SQL*Net roundtrips to/from client
 3 sorts (memory)
 0 sorts (disk)
 0 rows processed
```

Usando un cluster: plan de ejecución de
`select * from editions where pub_place='Barataria';`

En el caso de `pub_place='Barataria'` sucede algo similar a `pub_place='Segovia'`, al no devolver ninguna fila. Viendo los resultados se observa una clara mejora en los *consistent gets* utilizando el índice.

Finalmente, pese a que ambos mejoran el coste de la consulta, tras analizar los tests (lo que se verá en el apartado 4), nos decantamos por el índice.

- **Consulta 2:** select * from editions where publisher='...';

Creación de un índice:

Como se ha mencionado anteriormente, en estos casos la implementación de un índice es una gran mejora para llegar a un código lo más optimizado posible. En los dos primeros casos esto es porque se devuelve un número de filas bastante bajo, por lo que acceder directamente usando un índice reduce considerablemente el número de accesos. Sucede algo parecido en el último caso, ya que en este no se devuelven filas. Para crear el índice se ha usado el siguiente código:

create index idx_ed2 on editions(publisher);

A continuación comprobamos si ha habido mejoras en los planes de ejecución de las consultas usando el índice como esperamos:

<pre>SQL> select * from editions where publisher='B'; 12 rows selected. Execution Plan ----- Plan hash value: 1358544071 Id Operation Name Rows Bytes Cost (%CPU) Time ---- - 0 SELECT STATEMENT 8 1752 11 (0) 00:00:01 1 TABLE ACCESS BY INDEX ROWID BATCHED EDITIONS 8 1752 11 (0) 00:00:01 * 2 INDEX RANGE SCAN IDX_ED2 8 3 (0) 00:00:01 Predicate Information (identified by operation id): ----- 2 - access("PUBLISHER"='B') Statistics ----- 20 recursive calls 0 db block gets 64 consistent gets 20 physical reads 0 redo size 3688 bytes sent via SQL*Net to client 372 bytes received via SQL*Net from client 2 SQL*Net roundtrips to/from client 0 sorts (memory) 0 sorts (disk) 12 rows processed</pre>	<pre>SQL> select * from editions where publisher='SM'; 1704 rows selected. Execution Plan ----- Plan hash value: 1358544071 Id Operation Name Rows Bytes Cost (%CPU) Time ---- - 0 SELECT STATEMENT 1329 284K 1274 (0) 00:00:01 1 TABLE ACCESS BY INDEX ROWID BATCHED EDITIONS 1329 284K 1274 (0) 00:00:01 * 2 INDEX RANGE SCAN IDX_ED2 1329 7 (0) 00:00:01 Predicate Information (identified by operation id): ----- 2 - access("PUBLISHER"='SM') Statistics ----- 1 recursive calls 0 db block gets 1665 consistent gets 1530 physical reads 0 redo size 355904 bytes sent via SQL*Net to client 1612 bytes received via SQL*Net from client 115 SQL*Net roundtrips to/from client 0 sorts (memory) 0 sorts (disk) 1704 rows processed</pre>
---	--

Usando un índice: Plan de ejecución de
select * from editions where publisher='B';

Usando un índice: Plan de ejecución de
select * from editions where publisher='SM';

```
SQL> select * from editions where publisher='C';
no rows selected

Execution Plan
-----
Plan hash value: 1358544071

   Id | Operation                | Name      | Rows  | Bytes | Cost (%CPU)| Time     |
---- | -
0  | SELECT STATEMENT          |           |      8 | 1752 | 11 (0) | 00:00:01 |
1  | TABLE ACCESS BY INDEX ROWID BATCHED | EDITIONS |      8 | 1752 | 11 (0) | 00:00:01 |
* 2  | INDEX RANGE SCAN          | IDX_ED2   |      8 |      | 3 (0) | 00:00:01 |

Predicate Information (identified by operation id):
-----
   2 - access("PUBLISHER"='C')

Statistics
-----
       1 recursive calls
       0 db block gets
       3 consistent gets
       2 physical reads
       0 redo size
    1467 bytes sent via SQL*Net to client
     357 bytes received via SQL*Net from client
       1 SQL*Net roundtrips to/from client
       0 sorts (memory)
       0 sorts (disk)
       0 rows processed
```

Usando un índice: Plan de ejecución de
select * from editions where publisher='C';

Efectivamente, podemos ver que hay una muy notable mejora en la cantidad de *consistent gets* si lo comparamos con las estadísticas de antes de introducir el índice, tal y como se esperaba.

- **Consulta 3:** select * from copies where condition='...';

Creación de un índice:

En este caso, la creación de un índice no supone ninguna mejora, puesto que no usa el índice a no ser que se fuerce con hints.

De hecho, empeora el resultado, ya que, al no hacer una mejora, el programa no usa el índice de manera automática y se debe forzar su uso con hints. Para crear el índice se ha usado el siguiente código:

create index idx_ed3 on copies(condition);

<pre>SQL> select /*+ index(copies) */ * from copies where condition='G'; 48329 rows selected. Execution Plan ----- Plan hash value: 2073975869 Id Operation Name Rows Bytes Cost (%CPU) Time ---- - 0 SELECT STATEMENT (1) 00:00:01 1 TABLE ACCESS BY INDEX ROWID BATCHED COPIES 53349 2248K 1047 (1) 00:00:01 * 2 INDEX RANGE SCAN IDX_ED3 93 (2) 00:00:01 Predicate Information (identified by operation id): ----- 2 - access("CONDITION"='G') Note ----- - dynamic statistics used: dynamic sampling (level=2) Statistics ----- 58 recursive calls 0 db block gets 7516 consistent gets 95 physical reads 0 redo size 1993172 bytes sent via SQL*Net to client 35818 bytes received via SQL*Net from client 3223 SQL*Net roundtrips to/from client 3 sorts (memory) 0 sorts (disk) 48329 rows processed</pre>	<pre>SQL> select /*+ index(copies) */ * from copies where condition='N'; 48361 rows selected. Execution Plan ----- Plan hash value: 2073975869 Id Operation Name Rows Bytes Cost (%CPU) Time ---- - 0 SELECT STATEMENT (1) 00:00:01 1 TABLE ACCESS BY INDEX ROWID BATCHED COPIES 51984 2182K 1021 (1) 00:00:01 * 2 INDEX RANGE SCAN IDX_ED3 90 (0) 00:00:01 Predicate Information (identified by operation id): ----- 2 - access("CONDITION"='N') Note ----- - dynamic statistics used: dynamic sampling (level=2) Statistics ----- 55 recursive calls 0 db block gets 7523 consistent gets 86 physical reads 0 redo size 1995309 bytes sent via SQL*Net to client 35851 bytes received via SQL*Net from client 3226 SQL*Net roundtrips to/from client 3 sorts (memory) 0 sorts (disk) 48361 rows processed</pre>
--	--

Usando un índice: Plan de ejecución de
select * from copies where condition='G';

Usando un índice: Plan de ejecución de
select * from copies where condition='N';

<pre>SQL> select /*+ index(copies) */ * from copies where condition='D'; 48240 rows selected. Execution Plan ----- Plan hash value: 2073975869 Id Operation Name Rows Bytes Cost (%CPU) Time ---- - 0 SELECT STATEMENT (1) 00:00:01 1 TABLE ACCESS BY INDEX ROWID BATCHED COPIES 51714 2171K 1016 (1) 00:00:01 * 2 INDEX RANGE SCAN IDX_ED3 90 (0) 00:00:01 Predicate Information (identified by operation id): ----- 2 - access("CONDITION"='D') Note ----- - dynamic statistics used: dynamic sampling (level=2) Statistics ----- 14 recursive calls 0 db block gets 7407 consistent gets 83 physical reads 0 redo size 1990153 bytes sent via SQL*Net to client 35752 bytes received via SQL*Net from client 3217 SQL*Net roundtrips to/from client 0 sorts (memory) 0 sorts (disk) 48240 rows processed</pre>
--

Usando un índice: Plan de ejecución de
select * from copies where condition='D';

Por tanto, como se muestra en las imágenes, la introducción de un índice no es una decisión adecuada para tratar de obtener la mejor optimización posible, puesto que estaríamos ocupando espacio innecesariamente con algo que no se llega a usar.

- **Consulta 4:** select * from editions;

En este caso, como mencionamos en el apartado 2, puesto que no tenemos una condición de búsqueda, no hay nada por lo que no interese filtrar usando un índice. Podríamos quizás pensar en usar la clave primaria como clave por la que filtrar, pero eso sería un sinsentido, ya que al realizar la creación de tablas, todas las claves que son *primary keys* crean automáticamente un índice.

Puesto que no podemos mejorar el rendimiento de la consulta mediante la creación de índices ni clusters hemos recurrido a la modificación del diseño físico de las tablas. Por los motivos ya expuestos, hemos reducido el espacio libre distribuido de los cubos (PCTFREE) para reducir el número de accesos necesarios al realizar el *full scan*.

```
SQL> select * from editions;
240632 rows selected.

Execution Plan
-----
Plan hash value: 1741989577

   Id | Operation          | Name    | Rows  | Bytes | Cost (%CPU)| Time     |
---- | -
   0  | SELECT STATEMENT    |         |       |       |          |         |
   1  | TABLE ACCESS FULL | EDITIONS | 230K  | 217M  | 1953  (1) | 00:00:01 |

Note
-----
- dynamic statistics used: dynamic sampling (level=2)

Statistics
-----
      0 recursive calls
      0 db block gets
    22700 consistent gets
      0 physical reads
      0 redo size
  57147402 bytes sent via SQL*Net to client
  176810 bytes received via SQL*Net from client
   16844 SQL*Net roundtrips to/from client
      0 sorts (memory)
      0 sorts (disk)
  240632 rows processed
```

Plan de ejecución de select * from editions;
 con 5% de PCTFREE

```
CREATE TABLE Editions(
  ISBN          VARCHAR2(20),
  TITLE         VARCHAR2(200) NOT
  NULL,
  AUTHOR        VARCHAR2(100) NOT
  NULL,
  LANGUAGE      VARCHAR2(50)
  default('Spanish') NOT NULL,
  ALT_LANGUAGES VARCHAR2(50),
  EDITION       VARCHAR2(50),
  PUBLISHER     VARCHAR2(100),
  EXTENSION     VARCHAR2(50),
  SERIES        VARCHAR2(50),
  COPYRIGHT     VARCHAR2(20),
  PUB_PLACE     VARCHAR2(50),
  DIMENSIONS    VARCHAR2(50),
  PHY_FEATURES  VARCHAR2(200),
  MATERIALS     VARCHAR2(200),
  NOTES         VARCHAR2(500),
  NATIONAL_LIB_ID VARCHAR2(20)
  NOT NULL,
  URL           VARCHAR2(200),
  CONSTRAINT pk_editions PRIMARY
  KEY(isbn),
  CONSTRAINT uk_editions UNIQUE
  (national_lib_id),
  CONSTRAINT fk_editions_books
  FOREIGN KEY(title,author)
  REFERENCES books(title,author)
)PCTFREE 5;
```

Efectivamente, observamos que se produce una reducción del número de *consistent gets* en relación a la consulta original con un PCTFREE mayor.

4 Evaluación

Como evaluación final veremos qué resultados nos da la ejecución de los tests que se encuentran en el paquete proporcionado. Con los resultados obtenidos decidiremos cuál es la mejor combinación para llegar al resultado final más optimizado que podemos alcanzar de acuerdo a las decisiones que hemos tomado durante el análisis.

Empezamos comprobando la eficiencia de los índices por separado para ver cuál es el más eficiente actuando por sí solo.

Consulta 1: solo usando el índice idx_ed1:

```
SQL> exec pkg_costes.run_test(10);  
Iteration 1  
Iteration 2  
Iteration 3  
Iteration 4  
Iteration 5  
Iteration 6  
Iteration 7  
Iteration 8  
Iteration 9  
Iteration 10  
RESULTS AT 01/05/2025 17:18:25  
TIME CONSUMPTION (run): 1014.1 milliseconds.  
CONSISTENT GETS (workload):45479 acc  
CONSISTENT GETS (weighted average):4547.9 acc  
  
PL/SQL procedure successfully completed.
```

Consulta 1: solo usando el cluster idx_ed1:

```
SQL> exec pkg_costes.run_test(10);  
Iteration 1  
Iteration 2  
Iteration 3  
Iteration 4  
Iteration 5  
Iteration 6  
Iteration 7  
Iteration 8  
Iteration 9  
Iteration 10  
RESULTS AT 01/05/2025 17:45:12  
TIME CONSUMPTION (run): 1170.4 milliseconds.  
CONSISTENT GETS (workload):57146 acc  
CONSISTENT GETS (weighted average):5714.6 acc  
  
PL/SQL procedure successfully completed.
```

Consulta 2: solo usando el índice idx_ed2:

```
SQL> exec pkg_costes.run_test(10);  
Iteration 1  
Iteration 2  
Iteration 3  
Iteration 4  
Iteration 5  
Iteration 6  
Iteration 7  
Iteration 8  
Iteration 9  
Iteration 10  
RESULTS AT 01/05/2025 17:20:23  
TIME CONSUMPTION (run): 1078.5 milliseconds.  
CONSISTENT GETS (workload):39393 acc  
CONSISTENT GETS (weighted average):3939.3 acc  
  
PL/SQL procedure successfully completed.
```

Consulta 3: solo usando el índice idx_ed3:

```
SQL> exec pkg_costes.run_test(10);  
Iteration 1  
Iteration 2  
Iteration 3  
Iteration 4  
Iteration 5  
Iteration 6  
Iteration 7  
Iteration 8  
Iteration 9  
Iteration 10  
RESULTS AT 02/05/2025 19:14:50  
TIME CONSUMPTION (run): 1054.1 milliseconds.  
CONSISTENT GETS (workload):60447 acc  
CONSISTENT GETS (weighted average):6044.7 acc  
  
PL/SQL procedure successfully completed.
```


A continuación, comprobamos los efectos aislados (sin índices) de reducir el espacio libre distribuido.

Consulta 4: solo reduciendo el espacio libre (PCTFREE = 5):

```
SQL> exec pkg_costes.run_test(10);
Iteration 1
Iteration 2
Iteration 3
Iteration 4
Iteration 5
Iteration 6
Iteration 7
Iteration 8
Iteration 9
Iteration 10
RESULTS AT 02/05/2025 19:01:52
TIME CONSUMPTION (run): 1117.1 milliseconds.
CONSISTENT GETS (workload):57429 acc
CONSISTENT GETS (weighted average):5742.9 acc
PL/SQL procedure successfully completed.
```

Para la consulta 1 decidimos quedarnos con el índice, ya que, aunque el cluster suponía una gran mejora con respecto al índice para pub_place='Madrid', al pasar los tests observamos que el resultado final es mucho mejor usando el índice.

A priori, vemos que el índice incluido en la consulta 2 sobre la clave publisher es el cambio que supone una mayor optimización del diseño.

Además, como mencionamos en el apartado 3, el índice 3 no mejora nada el resultado, por lo que no lo tomaremos en cuenta para las siguientes combinaciones a realizar.

Ahora, vamos a comparar cuál es la combinación que más nos interesa de cara al diseño final, combinando los índices entre sí y viendo si aumentar el espacio libre también beneficia.

Índice idx_ed1 e índice idx_ed2:

```
SQL> exec pkg_costes.run_test(10);
Iteration 1
Iteration 2
Iteration 3
Iteration 4
Iteration 5
Iteration 6
Iteration 7
Iteration 8
Iteration 9
Iteration 10
RESULTS AT 01/05/2025 17:19:28
TIME CONSUMPTION (run): 981.3 milliseconds.
CONSISTENT GETS (workload):24347 acc
CONSISTENT GETS (weighted average):2434.7 acc
PL/SQL procedure successfully completed.
```

Los dos índices y la reducción del espacio libre:

```
SQL> exec pkg_costes.run_test(10);
Iteration 1
Iteration 2
Iteration 3
Iteration 4
Iteration 5
Iteration 6
Iteration 7
Iteration 8
Iteration 9
Iteration 10
RESULTS AT 02/05/2025 19:52:32
TIME CONSUMPTION (run): 924.7 milliseconds.
CONSISTENT GETS (workload):23390 acc
CONSISTENT GETS (weighted average):2339 acc
PL/SQL procedure successfully completed.
```

Finalmente, ejecutando los tests para las distintas combinaciones de los cambios que hemos probado sobre la base de datos, podemos observar que la última situación es la que menos accesos realiza.

Por tanto, nuestra propuesta final de mejora es la creación de los índices 1 y 2 y la reducción del espacio libre distribuido.

5 Conclusiones Finales

Para concluir el trabajo, podemos afirmar que se ha realizado una optimización bastante buena, puesto que se ha conseguido reducir el número de accesos totales a algo más de un tercio de los accesos iniciales. Para conseguir un diseño incluso más optimizado se podría plantear modificar las tablas, añadiendo marcas de existencia, longitud o reiteración, así como modificar el *tablespace*. Sin embargo, en general, las impresiones sobre la práctica son buenas y consideramos que se ha conseguido una optimización aceptable.

En cuanto a las sensaciones con respecto a esta práctica, consideramos que ha sido sencilla de realizar, y agradecemos que se haya puesto al final del cuatrimestre, ya que de este modo no hemos tenido problemas para llegar a los tiempos de entrega fijados.

En relación al conjunto total de prácticas del cuatrimestre, pensamos que quizás hubiera sido de utilidad tener algún ejemplo de lo que se pedía, pero en general los materiales proporcionados eran buenos y suficientes. Los vídeos resultaron de utilidad para aprender a trabajar con SQL y estaba bien tenerlos disponibles siempre para poder acceder a ellos cuando lo necesitáramos.

Finalmente, ya cerrando con la asignatura en general, creemos que las clases estaban bien organizadas, aunque sentimos que se dedicó demasiado tiempo practicando álgebra relacional y faltó algo de práctica en otros aspectos de la asignatura, especialmente en el apartado de ficheros y diseño físico.